**Marine Plastic Tracker**

FOUNDATIONS · FROM SCRATCH

# Satellites, datasets, algorithms — and how they connect.

Start at zero. Which satellite we use and who owns it, where the images live, how many we can get, where the training data came from, whether it is even the same satellite, what algorithms do the work, and the single path that runs from an empty folder to the system we have today.

- → What satellite, and who runs it?
- → Where are the images stored?
- → How many can we get at once?
- → Where did the datasets come from?
- → Same satellite as our live feed?
- → Does the provider give labels too?
- → What algorithms did we use?
- → Is preprocessing actually used?
- → How do the three connect?
- → Which papers, which gaps?
- → Where are we now?
- → What comes next?

---

**Companion document.** This one covers the *foundations* — satellite, data provenance, algorithms and the end-to-end flow. **Model_and_Data_Explainer.pdf** covers the *model* — architecture, training recipe and measured results. **Everything here was verified against the running source code**, not from memory.

**Prepared for:** project panel / viva · **Date:** 30 July 2026

SECTION 1

# The mental model — three worlds, one bridge

Almost every confusing question about this project dissolves once you see that there are **three separate worlds**, and that one specific thing bridges them.




> **The single most important fact in this whole project:** World 1 and World 2 are the **same satellite**. MARIDA, MADOS, RFO and PLP are *all* built from Sentinel-2 imagery — the exact same sensor whose pixels we download live today. The scientists did not fly their own satellite; they downloaded the same free Sentinel-2 images we do, and then spent months **hand-labelling** them. That is why a model trained on their pixels works on ours.

So the answer to "does the agency that gives us satellite access also give us the datasets?" is: **partly, and it is the most important half that they do *not* give.**

| | COPERNICUS / ESA GIVES US | RESEARCHERS GAVE US |
|---|---|---|
| **What** | Raw **pixels** — reflectance measurements of the Earth's surface, worldwide, free, forever, for anyone. | **Labels** — "this specific pixel, in this specific image, is marine debris." |
| **Cost** | Free (account required for pixel access). | Free to download; enormously expensive to create (expert months). |
| **Can a model learn from it alone?** | **No.** Pixels without answers teach nothing supervised. | **No.** Labels are useless without the pixels they point at. |
| **Together** | **Yes — labelled pixels are exactly what supervised learning needs.** This is why the project is possible at zero data-acquisition cost. | |

> **Why this matters for the panel.** If someone asks "how did you get labelled marine plastic data — did you go out on a boat?", the honest and correct answer is: **no, and nobody could afford to at this scale.** We stand on four published, peer-reviewed labelling efforts, cite all of them, and our contribution is everything *after* the labels: unifying four incompatible formats into one contract, training, calibration, and putting it into a live operational system.

SECTION 2

# Everything about the satellite we use

## Sentinel-2, in plain terms

**Sentinel-2** is a pair (now a small constellation) of Earth-observation satellites operated by the **European Space Agency (ESA)** under the European Union's **Copernicus** programme. It is the workhorse of free optical Earth observation, and it is the only sensor this project uses for detection.

| PROPERTY | VALUE | WHY IT MATTERS TO US |
|---|---|---|
| **Operator** | ESA / EU Copernicus programme | Public, civilian, non-commercial. |
| **Cost to us** | **₹0 — free and open** FREE | **The entire project runs at zero data cost. This is a deliberate design constraint, not a limitation we tolerate.** |
| **Orbit** | Sun-synchronous, ~786 km altitude | Crosses every location at roughly the same local time → consistent sun angle, so reflectance is comparable across dates. |
| **Swath width** | 290 km | One pass covers a very wide strip — regional-scale monitoring is feasible. |
| **Revisit** | **~5 days** at the equator (constellation) | **The hard operational limit.** We cannot watch a debris patch continuously. A patch seen today gets its next look in about five days — which is precisely why drift **forecasting** is a core feature and not a nice-to-have. |
| **Resolution** | **10 m** (some bands 20 m / 60 m) | **The hard scientific limit.** One pixel = 10×10 m = 100 m² of sea. Anything smaller than a few pixels is invisible. We can never see a bottle — only large rafts. |
| **Spectral bands** | 13, from visible to short-wave infrared | The infrared bands are the whole game — plastic separates from water in the infrared, not in visible colour. |
| **Sees through cloud?** | **No** — it is an optical sensor | Monsoon and heavy cloud simply block observation. We measure and report cloud honestly rather than guessing beneath it. |
| **Sees at night?** | **No** — it needs sunlight | Daytime passes only. |

## The five bands we actually use, and why exactly these

Sentinel-2 offers 13 bands. We download **five** — plus one non-optical quality layer. Each was chosen for a physical reason, and the wavelengths are hard-coded into our FDI formula.

| BAND | WAVELENGTH | NATIVE RES. | WHAT IT IS FOR |
|---|---|---|---|
| **B04** Red | 665 nm | 10 m | Visible reference; feeds NDVI and the Plastic Index. |
| **B06** Red-edge | 740 nm | 20 m | Lower anchor of the FDI baseline; separates vegetation/algae. |
| **B08** NIR | 842 nm | 10 m | **The key band.** Water absorbs near-infrared almost completely, so it looks black — anything floating on the surface reflects it and lights up. |
| **B8A** Narrow NIR | 865 nm | 20 m | Second infrared view; helps distinguish algae from plastic. |
| **B11** SWIR | 1610 nm | 20 m | Upper anchor of the FDI baseline; strongly absorbed by water. |
| **SCL** (not a band) | — | 20 m | Scene Classification Layer — ESA's own per-pixel verdict on cloud / shadow / water / land / snow / nodata. We use it as a quality mask. |

> **The physics in one sentence.** Clean water absorbs near-infrared almost perfectly, so healthy open sea appears **black** in band B08 — therefore **anything that reflects near-infrared back at the satellite is, by definition, not water**: it is something floating. The Floating Debris Index measures exactly how far above the expected water baseline a pixel's infrared sits. That is the entire physical foundation, and everything else is deciding *which kind* of floating thing it is.

## L1C versus L2A — and why we insist on L2A

Sentinel-2 data is published at two processing levels, and choosing correctly matters enormously:

| LEVEL | WHAT IT MEASURES | USABLE FOR US? |
|---|---|---|
| **L1C** Top-of-Atmosphere | Light arriving at the **satellite** — includes haze, aerosols and atmospheric scattering. | **No.** The atmosphere would change the readings from day to day, so the same patch of plastic would produce different numbers on different dates. |
| **L2A** Bottom-of-Atmosphere | Light leaving the **Earth's surface**, after ESA removes the atmospheric effect. Also ships the **SCL** quality layer. | **Yes — we use L2A exclusively.** Values are comparable across dates and places, which is exactly what a trained model requires. |

Our live request explicitly asks for `"type": "sentinel-2-l2a"`, and the datasets we train on are surface-reflectance products too (MADOS uses the ACOLITE marine atmospheric correction). Training and inference therefore see the same physical quantity.

SECTION 3

# Where the satellite images are stored — and how many we can get

> **The answer will surprise you: we do not store satellite images at all.** There is no image folder, no image database, no downloaded scene archive in this project. We **stream** pixels on demand, use them within seconds, and let them go. What we keep is the *result* — detections, hotspots, forecasts — never the imagery.

## Where the images actually live

| LAYER | WHERE | WHO PAYS / OWNS |
|---|---|---|
| The full Sentinel-2 archive | **Copernicus Data Space Ecosystem (CDSE)** — ESA's own cloud, petabytes, every scene since 2015. | ESA / EU. Free to access. |
| A mirror of the same archive | **AWS Open Data** (Element 84's Earth Search catalogue). | Amazon hosts it free as open data. |
| **Our project** | **Nothing on disk.** Pixels arrive as an in-memory NumPy array and are discarded after the scan. A **25-second** in-memory cache stops the UI re-fetching the same box while polling. | **Us — and it costs nothing because nothing is stored.** |
| What we *do* keep | Detections, hotspots, trajectories, alerts and scan metadata in **PostgreSQL**; the trained model as a 551 KB ONNX file. | Us. |

> **Why streaming is the right architecture, not a shortcut.** One Sentinel-2 scene is roughly a gigabyte. Storing enough of the Indian Ocean to be useful would mean terabytes, a storage bill, a sync pipeline, and stale data. Instead ESA already stores it perfectly and gives us free access — so our system holds **the answers, not the raw material**. As a bonus, every scan is automatically as fresh as the satellite allows. The one real cost is that we cannot scan offline.

## The two doors we use to reach the data

We use two *different* services for two *different* jobs — a distinction worth being precise about:

### Door 1 — AWS Earth Search (STAC)

**Job:** find out *what exists*.
**Returns:** metadata only — scene ID, MGRS tile, acquisition date, cloud-cover %, footprint, thumbnail.
**No pixels.**
**Auth: none** — completely keyless.
**In code:** `satellite/download.py → search_scenes()`, default **8** scenes per query.

### Door 2 — CDSE Sentinel Hub Process API

**Job:** get *the actual pixels*.
**Returns:** a float32 GeoTIFF of exactly the bands and area we ask for.
**Auth: OAuth2 client credentials** — a free account at `dataspace.copernicus.eu`; the token is cached until just before expiry.
**In code:** `satellite/band_loader.py → fetch_bands()`.

## How many images can we get at a time?

This needs a careful answer, because **we do not request "images" at all** — we request **an area**, and the server builds a custom composite for it.

| QUESTION | PRECISE ANSWER | WHERE IT IS SET |
|---|---|---|
| Scenes listed per search | **8** by default (a parameter, not a hard cap) | `search_scenes(limit=8)` |
| **Images returned per pixel request** | **Exactly one composite** — *not* one scene. The server takes every pass in our time window and builds one best image from them. | `mosaickingOrder: "leastCC"` |
| Maximum size of one request | Server allows 2,500 px per side; **we self-limit to 1,024** to bound cost and latency. | `_MAX_DIM = 1024` |
| Bigger areas | **Native-resolution tiling** — split into up to **9 tiles** (3×3), fetched separately and stitched into one array. | `_MAX_TILES = 9` |
| Very large regions | Rather than exploding into hundreds of tiles, it degrades to **one coarser request** — deliberately trading resolution for a bounded quota. | `fetch_bands()` |
| Ground area covered | At ~20 m/px, one tile ≈ **20 km** per side; 3×3 tiles ≈ **61 km** per side. | derived |
| Time window searched | Last **45 days**, automatically widened to **180 days** if coverage is poor. | `INGEST_DAYS_BACK` |
| Cloud ceiling | Scenes above **50 %** cloud are excluded outright. | `INGEST_MAX_CLOUD=50` |

**The "least-cloud composite" is a subtle idea worth explaining properly.** We do not pick one photograph. We tell the server: "*for this box, look at every Sentinel-2 pass in the last 45 days that was under 50 % cloud, and for each pixel give me the clearest available observation.*" The result is a single image built from possibly several dates. That is why a scan works even when no single pass was cloud-free — and also why we display a **reliability grade** (reliable / degraded / unusable) alongside every scan, computed from how much of the box was actually observed and what it turned out to be.

## Choosing 50 % cloud was tuned, not guessed

| SETTING | WHAT HAPPENED |
|---|---|
| 100 % (accept anything) | A monsoon scene returned **cloud-top false positives** — bright cloud edges have a high FDI. |
| 25 % (very strict) | Too few clear passes existed; most pixels came back empty, leaving **~18 % coverage**. |
| **50 % (shipped)** | **Full coverage at effectively the same low cloud, because least-cloud compositing already picks the clearest pixels — while the scene filter still rejects the worst passes.** |

SECTION 4

# Are the datasets from the same satellite?

Yes — and this is not a coincidence. It is the reason the project works at all, and we deliberately rejected data that would have broken it.

| DATASET | SATELLITE | PRODUCT LEVEL | COMPATIBLE WITH OUR LIVE FEED? |
|---|---|---|---|
| **MARIDA** | Sentinel-2 | Surface reflectance | **Yes** — identical sensor and bands |
| **MADOS** | Sentinel-2 | ACOLITE marine surface reflectance | **Yes** — bands stored per wavelength, re-sorted by our adapter |
| **RFO** | Sentinel-2 | Mixed L1C / L2A, 12–13 bands | **Yes**, after our adapter resolves bands per file and rescales |
| **PLP** | Sentinel-2 | L2A | **Yes** |
| **Our live scan** | Sentinel-2 | L2A via CDSE | — (this is the target) |

**All five are the same instrument.** A model trained on any of them can be applied to any other, provided the features are computed identically.

## The one dataset we own but deliberately did not use

**PLP 2022 ships PlanetScope imagery at 3 m resolution, and it is sitting on our disk unused.** Three-metre pixels would largely solve our sub-pixel blindness. We did not use it, for a reason that is worth stating: **PlanetScope has no short-wave infrared band**. No SWIR means no B11, which means **the FDI cannot be computed at all** — our entire feature contract, and the physics it encodes, does not exist on that sensor. Using it is not a data-loading task; it is **a different model, different features and a different project** — and PlanetScope is commercial, which breaks our zero-cost constraint. It is listed in Future Scope, honestly, rather than quietly ignored.

## What "compatible" actually required us to enforce

Same satellite is necessary but **not sufficient**. Four things had to be forced into agreement, and each was a real source of bugs:

| # | MUST MATCH | WHAT BROKE WHEN IT DIDN'T |
|---|---|---|
| 1 | **Band selection and order** — always `B04, B06, B08, B8A, B11` | MADOS names files by wavelength, MARIDA stacks by index, RFO varies 12/13 bands per file. Any mismatch silently feeds the model the wrong number in the wrong slot. |
| 2 | **Reflectance scale** — always 0–1 | Some products ship integers 0–10,000. Our PLP reader once mis-detected this and wrote `B11 = 158, FDI = −185` — physically impossible values — into a test set. |
| 3 | **Pixel grid** — everything on 10 m | Bands come at 10/20/60 m. Mixing resolutions in one formula compares different ground areas. |
| 4 | **Feature formulas** — byte-identical arithmetic | If training computes FDI one way and the live engine another, the model receives a different quantity than it learned, and fails **silently** — no crash, no error, just quietly wrong answers. |

**How we made #4 impossible to get wrong.** The feature computation exists as one function in the trainer ( `augment11()` ) and one mirrored function in the live engine ( `_features()` ), with the canonical order frozen as a constant ( `model_loader.FEATURE_NAMES` ). All three carry comments pointing at each other and stating that they must be changed together. This is the bridge between World 1 and World 2 — if it breaks, everything downstream is meaningless, so we designed it to be loud rather than subtle.

SECTION 5

# How we collected the datasets, step by step

Concretely, what a person actually did — in the order they did it.

**STEP 1**
### Read the papers
Find who has published labelled marine-debris imagery at all. The answer is: very few groups.

**STEP 2**
### Follow the DOI
Each paper links a Zenodo record or a university release page.

**STEP 3**
### Download
Archives, over a ~1 MB/s connection. MADOS alone is a 4.0 GB zip.

**STEP 4**
### Unpack
Into `data/<name>/`, left exactly as published — never edited in place.

**STEP 5**
### Inspect
Print the class histogram and mean spectra **before** trusting any documentation.

**STEP 6**
### Write an adapter
One Python reader per dataset, all emitting the identical 7-number contract.

**STEP 7**
### Cache
Compressed `.npz` arrays — 34 GB of imagery becomes 35 MB of features.

| DATASET | WHERE IT CAME FROM | SIZE ON DISK |
|---|---|---|
| **MARIDA** | Published with Kikaki et al. 2022 (*PLOS ONE* 17(1):e0262247) — public research release. | 4.5 GB |
| **MADOS** | Zenodo, DOI **10.5281/zenodo.10664073**, published with Kikaki et al. 2024 (*ISPRS JPRS*). | 5.6 GB |
| **RFO** | Public research release accompanying Rußwurm et al. 2023. | 23 GB |
| **PLP** | Plastic Litter Project, University of the Aegean (Topouzelis et al.) — campaign data for 2021 and 2022. | 79 MB |
| Durban scene | Auxiliary scene used with RFO. | 0.9 GB |
| **Total raw imagery collected** | | **~34 GB** |
| **What the classifier actually trains on after preprocessing** | | **34.9 MB** |

**Nothing here was scraped, purchased, or generated by us.** Every real dataset is a citable, peer-reviewed scientific artefact from a named research group, downloaded from its official publication record. This matters if anyone questions provenance: we can point at a DOI for each one.

SECTION 6

# Is data preprocessing actually used?

> **Yes — and it runs in two completely separate places.** This is the part most people miss. Preprocessing happens **once, offline, on the datasets** to create training arrays; and **again, live, on every single scan** to create inference inputs. If those two paths ever disagree, the model breaks silently. Keeping them identical is arguably the core engineering discipline of the whole project.

| STEP | OFFLINE (TRAINING DATA) | LIVE (EVERY SCAN) | SAME? |
|---|---|---|---|
| Get pixels | Read GeoTIFF from `data/` | HTTP fetch from CDSE | different *source* |
| Put on 10 m grid | Nearest-neighbour upsample 20/60 m bands | Server returns one uniform grid | **equivalent** |
| Select 5 bands | `B04 B06 B08 B8A B11` | `B04 B06 B08 B8A B11` | **identical** |
| Scale to 0–1 | Per-scene DN detection ÷ 10,000 | Server returns float32 reflectance | **equivalent** |
| Quality mask | Use only researcher-labelled pixels | Drop cloud/shadow/land/snow/nodata via SCL | different *purpose* |
| Compute FDI & NDVI | `_features_from_bands()` | `fdi_calculator` + `_features()` | **identical maths** |
| **Build 11 features** | `augment11()` | `inference_engine._features()` | **identical, mirrored line-for-line** |
| Drop invalid | `np.isfinite(...).all()` | SCL mask + finite check | **equivalent** |
| Standardise | Mean/std stored *inside* the model | Nothing to do — it is baked into the ONNX | **impossible to desync** |

## Live-only preprocessing that training never needs

- **SCL masking.** Training data is already curated by researchers; a live scan is raw reality, so cloud (3, 8, 9, 10), land (4, 5), snow (11) and nodata (0, 1) must be removed first. We keep only SCL **6 (water)** and **7 (unclassified)** — class 7 matters because 20 m debris rafts frequently land there.
- **Dropping SCL class 2 (dark area).** It used to be kept and was our single biggest land/haze false-positive leak. Removing it is a one-line config change with a large real effect.
- **The FDI candidate gate.** Only pixels with `FDI > 0.02` ever reach the model — millions of pixels reduce to a few hundred candidates.
- **Surface composition and reliability grading.** Every scan reports how much of the box was water, land, cloud, and actually observed at all, then grades itself *reliable / degraded / unusable*.

> **A real bug this discipline caught.** Our surface percentages were computed over the whole grid including **nodata** pixels. A least-cloud composite leaves nodata wherever no clear pass existed — so a genuinely half-land scene diluted to **"land 0 %"**. The fix was to compute composition over **observed pixels only** and report `coverage_pct` separately. Same imagery, same model, completely wrong readout — from a preprocessing detail nobody would notice without checking.

SECTION 7

# Every algorithm in the project

Not just the training algorithms — the complete list, in the order the data meets them. OURS = implemented by us from the paper or definition. LIBRARY = provided by a framework.

## A. Detection algorithms

| ALGORITHM | WHO | WHAT IT DOES, IN PLAIN WORDS |
|---|---|---|
| **Floating Debris Index**<br>Biermann et al. 2020 | OURS | Draws a straight line between the red-edge (740 nm) and short-wave infrared (1610 nm) readings — where water "should" sit — then measures how far above that line the near-infrared (842 nm) actually is. Floating material sits above the line. Pure arithmetic, no learning, fully interpretable. |
| **Spectral indices**<br>NDVI, PI, MNDWI, NDVI_RE, brightness | OURS | Five more ratios from the same five bands, each emphasising a different confuser — vegetation, water, foam, glint. |
| **Multi-Layer Perceptron**<br>the classifier itself | OURS arch<br>LIBRARY autograd | **A feed-forward neural network, 11 → 256 → 128 → 64 → 32 → 4. Each layer multiplies by a weight matrix and zeroes negatives (ReLU). Learns non-linear combinations of the 11 features that no hand-written rule could express.** |
| **Softmax + argmax** | OURS | Turns four raw scores into four probabilities summing to 1; the largest wins and its probability becomes the confidence. |
| **Ensemble averaging** | OURS | Three networks trained from different random starts; their raw outputs are averaged. Uncorrelated mistakes cancel out. |
| **Vision Transformer**<br>Clay v1.5, frozen | LIBRARY | A large pretrained geospatial foundation model. We never train it — we feed it a 64×64 chip and take the 1024-number "fingerprint" it produces, then compare fingerprints. |
| **Cosine similarity index** | OURS | Measures the angle between two fingerprints to answer "how similar is this patch to known plastic?" Written in **pure NumPy** specifically so the production engine never needs PyTorch installed. |

## B. Training algorithms

| ALGORITHM | WHO | WHAT IT DOES |
|---|---|---|
| **Backpropagation** | LIBRARY (and OURS once, by hand) | Computes how much each weight contributed to the error, via the chain rule. PyTorch does it in the shipped trainer — but we also wrote a complete **manual backprop implementation in pure NumPy** ( `ml/train.py` ) to prove we understand it. |
| **AdamW optimiser** | LIBRARY (hand-written in the NumPy baseline) | Decides how far to step each weight, adapting per-parameter using running averages of gradient and gradient-squared, with weight decay to discourage large weights. |
| **Cosine annealing LR** | LIBRARY | Starts with big learning steps and smoothly shrinks them to near zero — explore first, settle later. |
| **Focal loss** Lin et al. 2017 | OURS | **Scales each pixel's error by `(1 - p_correct)^1.5` so easy, abundant water pixels stop dominating the gradient and the network is forced to work on the hard rare classes.** |
| **Class-balanced weights** Cui et al. 2019 | OURS | **Weights each class by its "effective number of samples" rather than raw frequency — gentler than inverse-frequency, which had previously collapsed sea-foam precision to 1.6 %.** |
| **Label smoothing + dropout** | LIBRARY | Two anti-overconfidence measures: never demand a 100 % certain answer, and randomly switch off neurons during training so no single path is relied upon. |
| **Coordinate-ascent bias calibration** | OURS | **After training, searches a small grid of per-class output offsets for the combination that maximises validation macro-F1, then folds the winner into the final layer's bias. Moves the decision boundary to the right place without any threshold logic at inference.** |
| **Best-epoch selection** | OURS | Score validation macro-F1 after every epoch, keep the best snapshot, discard the rest. Prevents shipping an overfitted final epoch. |

## C. Post-detection algorithms

| ALGORITHM | WHO | WHAT IT DOES |
|---|---|---|
| **DBSCAN clustering** Ester et al. 1996 | OURS from scratch, no scikit-learn | Groups scattered plastic pixels into **hotspots**. Any point with at least **3** neighbours within **0.09°** seeds a cluster, which then grows outward through its neighbours. Points that never qualify are labelled **noise** and dropped — which is exactly the behaviour we want: isolated single detections should not dispatch a boat. |
| **Severity scoring** | OURS | Detections per hotspot → HIGH (≥10) / MEDIUM (≥5) / LOW (≥3). Only HIGH and MEDIUM raise alerts. |
| **Lagrangian drift, RK4** | OURS | **Predicts where a patch travels over 72 hours. It advances a virtual parcel one hour at a time using 4th-order Runge–Kutta — which samples the current field four times per step instead of once — because simple Euler steps drift badly off curved ocean currents.** |
| **Windage ensemble** | OURS | Floating plastic is pushed by wind as well as current, and how much depends on how high it floats. We run a spread of windage values (1.5–3 %) and output a **cone of uncertainty, not a single line** — an honest forecast, not a false-precision one. |
| **Liu–Weisberg skill score** Liu & Weisberg 2011 | OURS | The standard way to score a predicted trajectory against an observed one. **The harness exists; we have not yet run real drifter tracks through it** — so drift ETAs are labelled as unvalidated estimates. |

> **Why write DBSCAN and RK4 by hand instead of importing them?** Two reasons, both practical. First, the production engine is deliberately kept to **numpy + onnxruntime only** — no scikit-learn, no SciPy, no PyTorch — so it starts fast, installs anywhere and has a small dependency surface. Second, we can explain every line to a panel, which we could not honestly do for an imported black box.

SECTION 8

# The complete flow — from an empty folder to today

If someone had to rebuild this project from zero, this is the exact order in which it would have to happen. Phases 1–4 are done once; phase 5 runs every time an analyst presses Scan.

```
╔═══ PHASE 1 · UNDERSTAND THE PROBLEM ═══════════════════════════════════╗
  Read the literature → learn that water absorbs near-infrared, so floating
  matter is visible in NIR → find the Floating Debris Index (Biermann 2020)
  → discover the ceiling: 10 m pixels, ~5-day revisit, no cloud penetration.
╚═══════════════════════════════════════════════════════════════════════╝
                                    ▼
╔═══ PHASE 2 · GET LABELLED DATA ════════════════════════════════════════╗
  Find published labelled Sentinel-2 debris data (there is very little)
  → download MARIDA, MADOS, RFO, PLP  (~34 GB)
  → unpack into data/, untouched.
╚═══════════════════════════════════════════════════════════════════════╝
                                    ▼
╔═══ PHASE 3 · BUILD THE BRIDGE (preprocessing) ═════════════════════════╗
  Decide the contract: 5 bands → 7 features → 11 features, fixed order
  → write ONE adapter per dataset, all emitting that contract
  → verify class meanings by mean spectra, not by documentation
  → map 15 research classes → 4 operational classes
  → respect official / scene-disjoint / date-disjoint splits
  → cache to .npz   (34 GB  →  34.9 MB)
╚═══════════════════════════════════════════════════════════════════════╝
                                    ▼
╔═══ PHASE 4 · TRAIN AND PROVE ══════════════════════════════════════════╗
  Train the MLP on GPU (focal loss + class-balanced weights + calibration)
  → select the best epoch by VALIDATION macro-F1
  → 3 seeds, average the logits
  → export to ONNX, prove parity = 1.0
  → score once on 499,907 untouched test pixels  →  macro-F1 0.818
  → test on data never trained on: RFO new seas, PLP deployed targets
  → publish the failures too.
╚═══════════════════════════════════════════════════════════════════════╝
                                    ▼
╔═══ PHASE 5 · RUN IT LIVE (every scan, ~seconds) ═══════════════════════╗
  Analyst draws a box
     → STAC search: which scenes exist, how cloudy   (keyless)
     → CDSE Process API: least-cloud composite, 5 bands + SCL   (OAuth)
     → mask cloud / land / shadow / nodata via SCL
     → FDI > 0.02  → millions of pixels become a few hundred candidates
     → build the SAME 11 features
     → marinenet.onnx → class + confidence per candidate
     → keep HIGH-confidence plastic (P ≥ 0.648, ~90% precise)
     → DBSCAN → hotspots → severity
     → Clay verifier → CONFIRMED / POSSIBLE
     → RK4 drift, 72 h, windage cone → landfall ETA
     → alerts, map, exports; results to PostgreSQL — imagery discarded
╚═══════════════════════════════════════════════════════════════════════╝
```

### How the three worlds touch, one more time

| CONNECTION | WHAT ACTUALLY LINKS THEM |
| --- | --- |
| **Satellite → Datasets** | Researchers downloaded Sentinel-2 scenes and hand-labelled the pixels. The imagery is the same public archive we use; the labels are their contribution. |
| **Datasets → Model** | Our adapters convert every dataset into the identical 11-feature contract; the trainer learns the mapping from those 11 numbers to 4 classes. |
| **Model → Satellite** | The live engine computes **the same 11 numbers** from a fresh CDSE fetch and feeds them to the trained file. **This only works because step 1 and step 3 use identical maths** — which is why the feature contract is guarded so carefully. |

SECTION 9

# The papers we read, and the gaps we found

## Foundational reading

| WORK | WHAT WE TOOK FROM IT | WHERE IT LIVES IN OUR CODE |
|---|---|---|
| **Biermann et al. 2020**<br>Finding plastic patches in coastal waters using optical satellite data · *Scientific Reports* | The Floating Debris Index itself, and the finding that Sentinel-2 *can* see plastic aggregations at all. This paper is the reason the project exists. | `satellite/fdi_calculator.py` |
| **Kikaki et al. 2022**<br>MARIDA · *PLOS ONE* 17(1):e0262247 | The first properly labelled benchmark with official splits, plus the insight that debris must be distinguished from *other* floating matter, not just from water. | `ml/datasets/marida.py` |
| **Kikaki et al. 2024**<br>MADOS · *ISPRS J. Photogramm. Remote Sens.* | A much larger successor benchmark; the volume that made a deep model viable. | `ml/datasets/mados.py` |
| **Rußwurm et al. 2023**<br>RefinedFloatingObjects | Scene-disjoint evaluation as the honest test of generalisation, and globally diverse water negatives. | `ml/datasets/rfo.py` |
| **Topouzelis et al.**<br>Plastic Litter Project, Univ. of the Aegean | The only true ground truth in the field — physically deployed targets. Also the source of our most important negative result. | `ml/datasets/plp.py`, `ml/validate_plp.py` |
| **Lin et al. 2017** · Focal loss<br>**Cui et al. 2019** · Class-balanced loss | The two techniques that made a 1,134 : 1 class imbalance trainable. | `ml/train_perpixel_gpu.py` |
| **Ester et al. 1996** · DBSCAN<br>**Liu & Weisberg 2011** · Skill score | Density clustering with an explicit noise class; the standard trajectory-validation metric. | `hotspot/dbscan_clusterer.py`, `ml/validate_drift.py` |
| **Clay v1.5**<br>open geospatial foundation model | A pretrained backbone generalises across seas far better than a small CNN trained from scratch on a few hundred scenes. | `ml/clay_*.py` |

## The gaps we found — and what we did about each

| GAP IN THE FIELD | OUR RESPONSE |
|---|---|
| **1. Papers detect; nothing operates.** The literature produces accuracy tables. Almost none produce something an agency could actually use on a Monday morning — no console, no dispatch, no drift forecast, no audit trail. | **This is our primary contribution.** An end-to-end operational console: scan → detect → cluster → forecast → alert → dispatch → export, with role-based access and a stored history. |
| **2. Detection without prediction.** Debris moves several kilometres a day, and the satellite returns in ~5 days. A detection alone is already stale when a crew reaches it. | 72-hour RK4 Lagrangian drift forecasting with a windage ensemble, producing a **cone** and a landfall ETA rather than a single misleading line. |
| **3. Metrics that flatter.** Marine-debris papers frequently headline accuracy on data that is ~98 % water, where "always water" scores 0.98. | Macro-F1 first, everywhere, in the product and in every document; accuracy always captioned as water-inflated; per-class numbers always shown. |
| **4. No confidence tiering.** A detection is usually presented as binary, which is useless for deciding whether to send a boat. | A two-tier system calibrated on held-out data: HIGH (~90 % precise) versus candidate. **Only HIGH-tier plastic can form a hotspot or dispatch a crew.** |
| **5. Negative results are not published.** Nobody reports the architecture that lost or the extra data that hurt. | We publish ours in-app: the patch CNN at 0.555, PLP at 0/33, the PLP retrain that cost 12 points of precision, and unvalidated drift. |
| **6. No closed loop.** Models are trained once and frozen; analyst knowledge never returns to the system. | An active-learning loop where analyst labels are captured and queued for retraining. |

SECTION 10

# Where we are now — and what comes next

## Honest status

| CAPABILITY | STATUS | EVIDENCE |
|---|---|---|
| Real satellite ingest | **Working** | Live CDSE OAuth fetch, native-resolution tiling, SCL masking, reliability grading |
| Trained classifier | **Working** | macro-F1 0.818 on 499,907 untouched pixels; ONNX parity 1.0 |
| Two-tier confidence | **Working** | 90.1 % precision measured at the shipped threshold |
| Hotspots + alerts | **Working** | Hand-written DBSCAN, severity thresholds, dispatch gating |
| Drift forecasting | **Working, unvalidated** | Real CMEMS/Open-Meteo forcing, RK4, windage cone — but no drifter tracks scored yet |
| Clay verifier | **Working** | GPU sidecar; fusion precision 0.92 |
| Active learning | **Loop closed** | Analyst labels captured and queued |
| Sub-pixel plastic | **Not solved — and not solvable at 10 m** | PLP 0/33, with the mechanism proven |

## Future scope, ordered by honest value

### 1. Validate the drift model against real drifter tracks HIGHEST VALUE, LOWEST COST

The harness already exists — `ml/validate_drift.py` with the Liu–Weisberg skill score. What is missing is **data**: real GPS drifter tracks (NOAA Global Drifter Program publishes them free). Feed a track's start point in, compare the predicted path against the observed one, publish the score. **This converts our single largest unvalidated claim into a measured one, costs nothing, and needs no new model.** If we had time for exactly one more thing, it is this.

### 2. Higher-resolution imagery for the sub-pixel problem

PlanetScope at 3 m would make a 14 m target span ~5 pixels instead of a fraction of one. **The honest catch, stated earlier:** PlanetScope has no SWIR band, so FDI cannot be computed and the entire feature contract must be redesigned — plus it is commercial. This is a new model, not an upgrade. Drone or aerial imagery for local validation is a cheaper variant of the same idea.

### 3. SAR fusion (Sentinel-1)

Radar penetrates cloud and works at night — directly attacking our worst operational limitation, the monsoon. Debris rafts dampen surface roughness, which radar can see. Combining Sentinel-1 radar with Sentinel-2 optical would give all-weather capability. Substantial work: different physics, different preprocessing, different features.

### 4. Multi-temporal analysis

Currently each scan is judged alone. But a real debris raft **moves consistently with the current** across passes, whereas a cloud artefact or sun-glint does not. Tracking candidates across the ~5-day cycle and rewarding physically plausible motion could raise precision using **only data we already fetch**.

### 5. Complete the active-learning loop end to end

Labels are already captured. The remaining step is automatic periodic retraining with the existing head-to-head promotion gate, so analyst corrections measurably improve the shipped model over time rather than sitting in a queue.

### 6. Field validation with an agency

The scientifically strongest possible next step: send a boat to a HIGH-confidence hotspot and record what is actually there. Even ten verified dispatches would give a real-world precision figure — something almost no paper in this field has.

**What we would *not* do, and why it matters that we know.** Download more 10 m labelled debris data expecting better plastic recall. **We tested this.** Adding PLP hurt precision (0.776 → 0.660) for two extra sub-pixel targets. More data at the same resolution cannot fix a resolution limit — knowing which lever is exhausted is as valuable as knowing which one to pull.

SECTION 11

# Straight answers to the questions that started this

### Which satellite do we use, and who runs it?

Sentinel-2, operated by the European Space Agency under the EU's Copernicus programme. Free and open. ~5-day revisit, 10 m pixels, 13 bands, 290 km swath. We use five bands plus ESA's SCL quality layer, at processing level L2A (surface reflectance).

### Where are the satellite images stored?

In ESA's Copernicus Data Space Ecosystem and its AWS Open Data mirror — **not in our project**. We stream pixels per scan into memory, use them, and discard them. Only results are stored, in PostgreSQL. There is no image folder anywhere in this repository.

### How many images can we get at a time?

A metadata search returns 8 scenes by default. A pixel request returns **one composite image** built from every qualifying pass in the last 45 days, capped at 1,024 px per side; larger areas are stitched from up to 9 native-resolution tiles (~61 km per side), and very large regions deliberately fall back to one coarser request to protect the free quota.

### Are the datasets from the same satellite as our live feed?

Yes — all four are Sentinel-2. That is exactly why a model trained on them can be applied to our live pixels. It is the single design fact the whole project rests on.

### Does Copernicus give us the datasets too?

No. Copernicus gives **pixels**, worldwide and free, with no labels whatsoever. The labels — "this pixel is marine debris" — came from four independent research groups who downloaded the same free imagery and hand-annotated it, then published it with a DOI. We collected those releases and wrote the adapters that make them mutually compatible.

### What algorithms did we use?

FDI and five spectral indices for physics-based screening; an MLP ensemble trained with backpropagation, AdamW, focal loss, class-balanced weights and coordinate-ascent bias calibration; DBSCAN for hotspots; RK4 Lagrangian advection with a windage ensemble for drift; and a frozen Vision Transformer with cosine similarity as a verifier. DBSCAN, RK4, the losses, the calibration and the similarity index are all written by us.

### Is data preprocessing actually used?

Extensively, in two separate places — offline to build training arrays from 34 GB of raw imagery, and live on every scan. Keeping those two paths numerically identical is the core engineering discipline of the project, and the feature computation is deliberately mirrored line-for-line between them.

### If someone started from scratch, what is the order?

Read the physics → find labelled data → build one common feature contract and the adapters that reach it → train, calibrate, and test on data you never touched → wrap it in a live ingest path that computes the identical features → cluster, forecast and act on the results. Section 8 is that path in full.

---

**Everything in this document was verified against source** while writing it: `satellite/band_loader.py` (CDSE OAuth, tiling, composite), `satellite/download.py` (STAC discovery, ingest orchestration), `satellite/cloud_masker.py` (SCL classes, reliability), `config.py` (all thresholds quoted), `ml/datasets/*.py` (dataset provenance and adapters), `ml/train_perpixel_gpu.py` (training algorithms), `hotspot/dbscan_clusterer.py` and `trajectory/lagrangian_model.py` (from-scratch implementations).

Companion document: **Model_and_Data_Explainer.pdf** — architecture, training recipe and measured results.